

Managed and external tables

Who owns the storage lifecycle, what DROP really does, and converting between the two with SET MANAGED.

The difference is not where the data sits. It is **who owns the storage lifecycle** — who decides when files are created, optimised and deleted.

The differences that matter

	MANAGED	EXTERNAL
Storage location	Unity Catalog decides	You specify with LOCATION
On DROP	Data files are deleted	Data files are left alone
Maintenance	Predictive optimisation handles it	You schedule it yourself
Access from other tools	Through Databricks	Directly at the path, if permitted
Status	The default and recommended type	For specific reasons

ON THE EXAM

Read the second row twice. It is the single most consequential difference and the one that appears in incident reports.

Creating each

- ◆ **CREATE TABLE** `catalog.schema.name (...)` with no **LOCATION** creates a managed table.
- ◆ Adding **LOCATION** 'path' makes it **external**, pointing at storage you control.
- ◆ The **LOCATION** must be covered by an **external location** in Unity Catalog, with a storage credential behind it.
- ◆ **CTAS works for both**: **CREATE TABLE ... AS SELECT**, with or without a **LOCATION**.

ON THE EXAM

The presence or absence of **LOCATION** is the whole distinction at creation time. There is no **MANAGED** keyword to remember.

What DROP does

- ◆ Dropping a managed table deletes the underlying data files.
- ◆ Dropping an external table removes the metadata only. The files remain at the path.
- ◆ **UNDROP TABLE can recover a dropped managed table** within the retention window.
- ◆ **But do not rely on it.** It is a safety net with a limit, not a substitute for care.

ON THE EXAM

An external table can be recreated over its files in seconds. A managed table's files are gone once the window closes.

Unity Catalog managed tables are now **the default and recommended table type**.
External tables are for specific reasons, not a neutral alternative.

When external is still the right choice

- ◆ **Another system writes the data**, and Databricks is a reader rather than the owner.
- ◆ **Non-Databricks tools must read the files directly** at their path.
- ◆ **The data already exists** and copying it is impractical or prohibited.
- ◆ **A contractual or regulatory requirement** fixes where the data must live.

ON THE EXAM

Notice the shape: external is right when something outside Databricks has a legitimate claim on the files.

ALTER TABLE ... SET MANAGED converts an external table to a managed one, keeping its name, history, permissions and views.

How SET MANAGED works

- ◆ Requires Databricks Runtime 17.0 or above, or serverless compute.
- ◆ All readers and writers must use name-based access, not path-based, as a prerequisite.
- ◆ Step one copies the data and the Delta log to the managed location, with no downtime.
- ◆ Step two switches over, moving commits made during step one and updating the metadata. Brief writer downtime here.

ON THE EXAM

If interrupted while copying, restart it and it continues from where it stopped. Do not run two conversions on one table at once.

After conversion

- ◆ **Read and write streams fail** and must be restarted with the same configurations.
- ◆ **Predictive optimisation is automatically enabled**, unless you had turned it off.
- ◆ **With predictive optimisation on, the data in the old external location is deleted after 14 days.**
- ◆ **From Databricks Runtime 18.1, path-based reads redirect** to the new managed location, so legacy code keeps working.

ON THE EXAM

That 14-day timer is the detail to carry. The old files are not kept indefinitely as a backup.

Rolling back

- ◆ **ALTER TABLE ... UNSET MANAGED** converts a managed table back to external.
- ◆ **Seven days after rollback, the data in the managed location is deleted automatically.**
- ◆ **Commits made between conversion and rollback allow time travel by version, but not by timestamp.**
- ◆ **Streaming jobs must be restarted** after a rollback, as after a conversion.

ON THE EXAM

Two timers to keep straight: 14 days deletes the old external data after conversion; 7 days deletes the managed data after rollback.

One order that matters

- ◆ You must run `UNSET MANAGED` before dropping a converted table.
- ◆ Dropping it without doing so may cause data loss or inconsistency.
- ◆ **The reason is the drop rule.** Dropping a managed table deletes its files, including the converted ones.
- ◆ **Check the table type with `DESCRIBE EXTENDED`** or Catalog Explorer before you act.

ON THE EXAM

Two rules from this module combine into a trap: converted tables are managed, and dropping a managed table deletes data.

Tiebreakers from this module

WHEN A QUESTION SAYS	REACH FOR
Dropping the table deleted the data	It was managed. Managed drops delete files.
Dropping the table left the files	It was external. Metadata only.
Convert external to managed	ALTER TABLE ... SET MANAGED, on DBR 17.0+ or serverless
Roll a converted table back	ALTER TABLE ... UNSET MANAGED
The old external files disappeared later	14 days after conversion, with predictive optimisation on
Another system writes these files	External is correct. Databricks is not the owner.
Which type is this table?	DESCRIBE EXTENDED, or Catalog Explorer

ON THE EXAM

Seven lines. The first two are the classic pair, and the fifth is the newest.

A team tidies up a schema, dropping eleven tables they believe are obsolete. Two turn out to be still needed. One is recreated in minutes by pointing a CREATE TABLE at its original path. The other cannot be recovered the same way.

Both tables were created by the same engineer, in the same schema, on the same day.

What is the difference between them?

- ☐ A The recoverable one was external, so DROP removed only its metadata; the other was managed, so its files were deleted
- ☐ B The recoverable one had been backed up and the other had not
- ☐ C The unrecoverable one was in a different catalog with stricter retention
- ☐ D DROP behaves differently depending on whether the table has been queried recently

◆ PAUSE AND DECIDE

What is the difference between them?

- A

 One was external, one was managed
Correct. The recovery method described — pointing CREATE TABLE at the original path — only works when the files were left in place, which is external behaviour.
- B

 One was backed up
Nothing suggests a backup, and the described recovery was recreating a table over an existing path, not restoring one.
- C

 A different catalog
The scenario states both were in the same schema, so they share a catalog.
- D

 Recency of querying
Invented behaviour. DROP does not consider query history.

THE SIGNAL

The recovery method is the evidence. Pointing CREATE TABLE at a path only works if the files are still there.

A team has forty external Delta tables they want to convert to managed tables, to get predictive optimisation. They need to keep table history, keep existing grants and views working, and minimise downtime for the streaming jobs that write to them.

An engineer proposes CREATE TABLE AS SELECT into new managed tables, then renaming.

What should they use instead?

- ☐ A The CTAS approach is fine and is the documented method
- ☐ B ALTER TABLE ... SET MANAGED, which retains history, permissions and views, and minimises downtime
- ☐ C DEEP CLONE into a managed table, then drop the originals
- ☐ D Move the files into the managed storage location manually

◆ PAUSE AND DECIDE

What should they use instead?

A CTAS is fine

It works, and it loses table history, creates a new object needing grants reapplied, and requires downtime while it copies.

C DEEP CLONE then drop

Also a real option Databricks names, and SET MANAGED minimises or eliminates downtime compared to it.

B ALTER TABLE ... SET MANAGED

Correct. Databricks recommends it over CTAS precisely for these reasons: history, name, settings, permissions and views retained, with minimal downtime.

D Move files manually

Unity Catalog manages the storage layout of managed tables. Hand-moving files leaves the metastore and the storage disagreeing.

THE SIGNAL

The team's three requirements — history, grants and views, low downtime — are the three benefits SET MANAGED is documented as providing.

Three weeks after converting a large external table with SET MANAGED, a data science team reports that a Python script has broken. The script reads the table by its old storage path rather than by name, and the path now returns nothing.

The cluster runs Databricks Runtime 16.4 LTS. Predictive optimisation is enabled.

What happened?

- ☐ A The conversion failed silently and should be re-run
- ☐ B With predictive optimisation on, the data in the old external location is deleted 14 days after conversion; the script needs to use name-based access
- ☐ C The table was rolled back with UNSET MANAGED by someone else
- ☐ D Path-based reads always fail against managed tables, from the moment of conversion

◆ PAUSE AND DECIDE

What happened?

- A

The conversion failed silently

It plainly succeeded — the table worked by name for three weeks. Only the old path stopped working.
- B

The 14-day deletion of the old external data

Correct. Three weeks is past 14 days, and path-based redirect requires DBR 18.1 or above, which this cluster is not on.
- C

Someone rolled it back

A rollback would make the table external again and the managed data would be deleted after 7 days — a different symptom on a different timer.
- D

Path-based reads always fail immediately

They worked for roughly two weeks after conversion, which is exactly why the failure was delayed and confusing.

THE SIGNAL

The delay is the clue. Nothing broke at conversion, and something broke three weeks later — which points at a timer, not an error.

A team converted an external table to managed six months ago. They now want to decommission it entirely and free the storage. An engineer plans to run DROP TABLE, reasoning that the original files are in their own storage account and will therefore survive. A colleague stops him.

Why?

- ☐ A The table is now managed, so DROP deletes its data; and the original external files were deleted 14 days after conversion anyway
- ☐ B DROP TABLE is not permitted on converted tables
- ☐ C He must run OPTIMIZE before dropping to release the storage
- ☐ D Nothing is wrong; his reasoning is correct

◆ PAUSE AND DECIDE

Why?

- A

 It is managed now, and the old files are long gone
Correct. Both halves of his reasoning are out of date: the table type changed, and the original external files were removed after 14 days.
- B

 DROP is not permitted
It is permitted, which is the danger. Nothing stops the command; it simply does more than he expects.
- C

 Run OPTIMIZE first
OPTIMIZE compacts files. It has nothing to do with releasing storage on a drop.
- D

 His reasoning is correct
It was correct before the conversion. Conversion changed the table type, and with it what DROP does.

THE SIGNAL

He is applying a true fact about the table's past. Databricks says to run UNSET MANAGED before dropping a converted table.

Managed means Unity Catalog owns the lifecycle — including deleting the files when you drop the table. **Conversion changes which rules apply.**